

AI Runtime OS (Working Title)

로컬 AI를 위한 운영체제

"GPU, 메모리, 컨텍스트, 에이전트 실행을 관리하여 AI가 현대 OS만큼 효율적으로 하드웨어를 활용할 수 있게 하는 실행 런타임"

목차

- 비전
- 문제 정의
- 현재 AI 스택 vs 제안 아키텍처
- 핵심 철학
- 주요 책임 영역
- 요청 실행 흐름
- 구현 현황 (cursor-local-llm)
- 로드맵
- 경쟁 환경
- 비즈니스 모델
- 장기 비전
- 부록: 효율화 전략

1. 비전

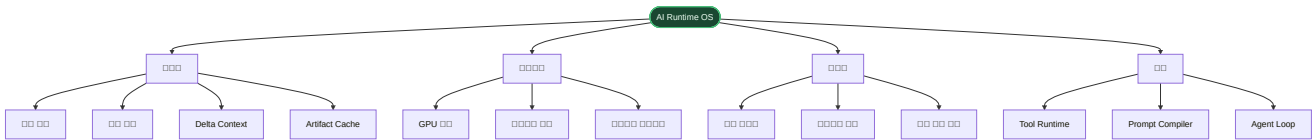
LLM의 능력은 빠르게 성장하고 있지만, 이를 실행하는 소프트웨어 스택은 거의 진화하지 못했다.

오늘날 AI 애플리케이션은 개발자에게 다음을 수동으로 맡긴다.

영역	개발자가 직접 다루는 것
GPU 메모리	VRAM 한도, OOM 대응
컨텍스트	프롬프트 길이, history 관리
KV Cache	할당 위치, offload 정책
멀티 GPU	tensor split, affinity
양자화	Q4/Q8, 모델 포맷 선택
모델 라우팅	작업별 모델 전환
에이전트 메모리	세션·장기 기억
프롬프트 최적화	system prompt, tool format

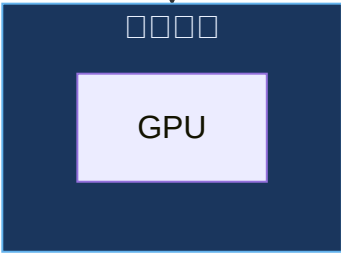
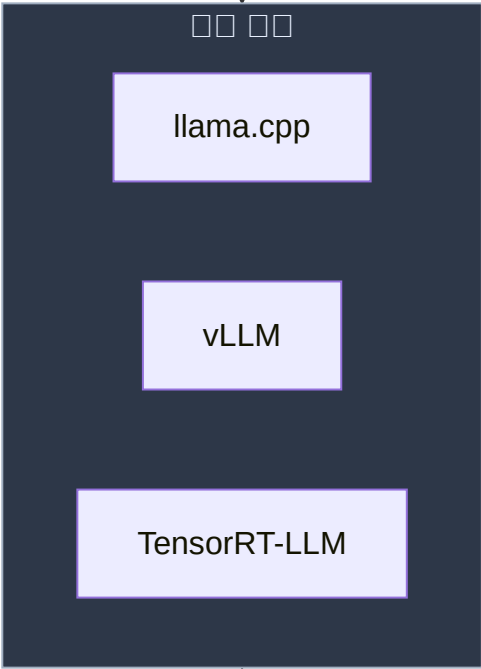
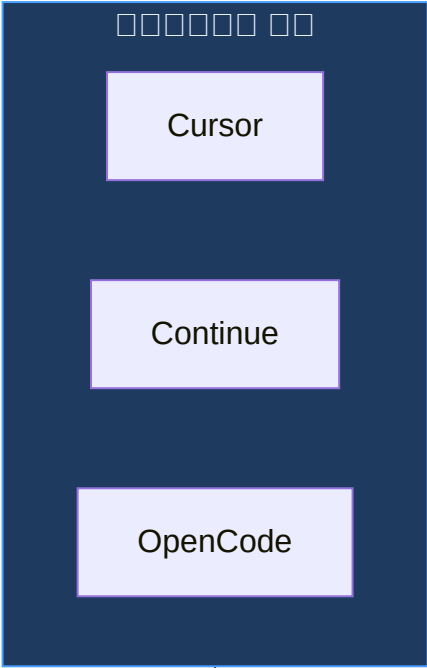
그 결과 **비싼 하드웨어는 낭비**되고, 개발자는 제품이 아닌 **인프라 튜닝**에 시간을 쓴다.

AI Runtime OS는 AI 애플리케이션과 추론 엔진 사이의 **운영체제**가 되어 이 문제를 해결한다.



2. 문제 정의

2.1 오늘의 스택 — 추론만 담당



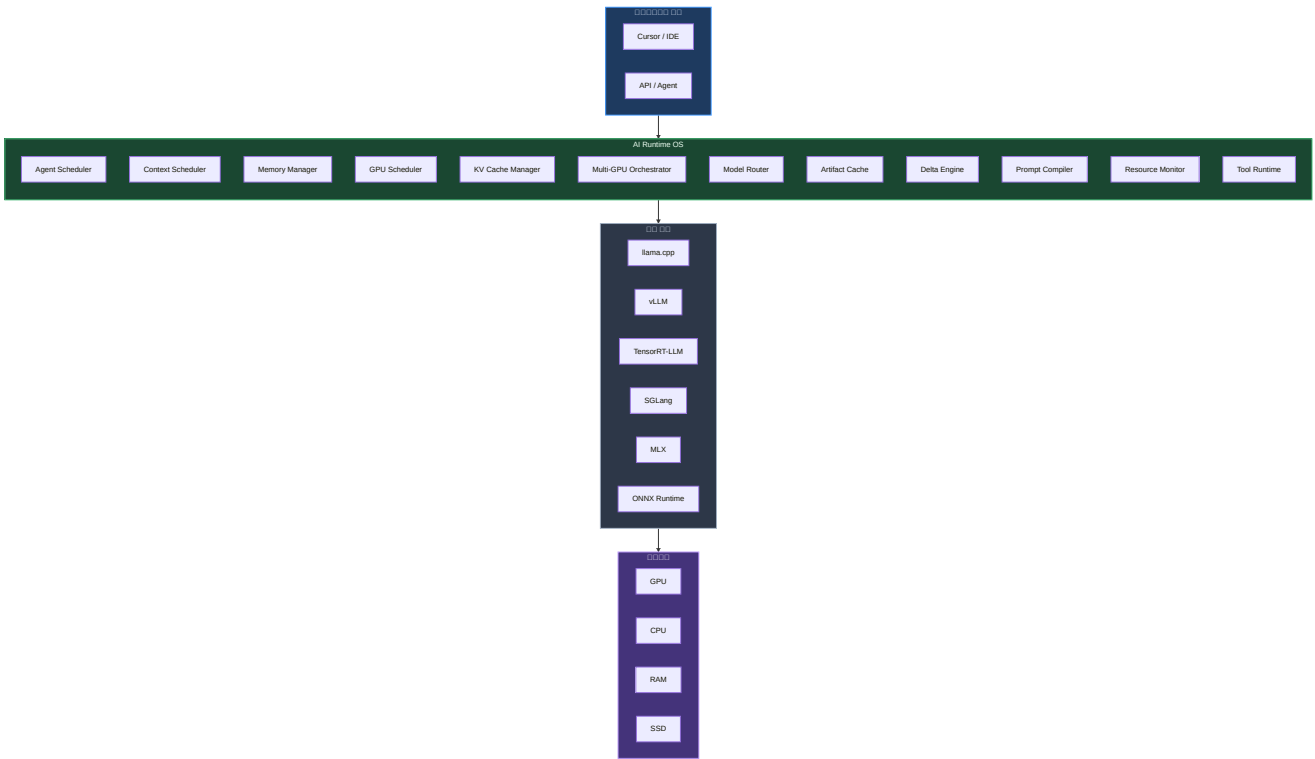
핵심 한계: 전체 AI 워크로드를 관리하는 시스템이 없다.
각 요청은 "또 하나의 프롬프트"로 취급되며, 스케줄링·메모리·컨텍스트 최적화는 앱 또는 개발자 몫이다.

2.2 실제로 겪는 증상

증상	원인
100K+ history → OOM	컨텍스트가 무한 성장
Agent loop 중단	stream/tool_call 포맷 불일치
GPU 2대인데 1대만 사용	수동 tensor split, 프로필 미분리
같은 모델 2번 로드	fast/long 중복 VRAM
느린 응답	불필요한 prefill (전체 history 재전송)

3. 현재 AI 스택 vs 제안 아키텍처

3.1 제안 아키텍처



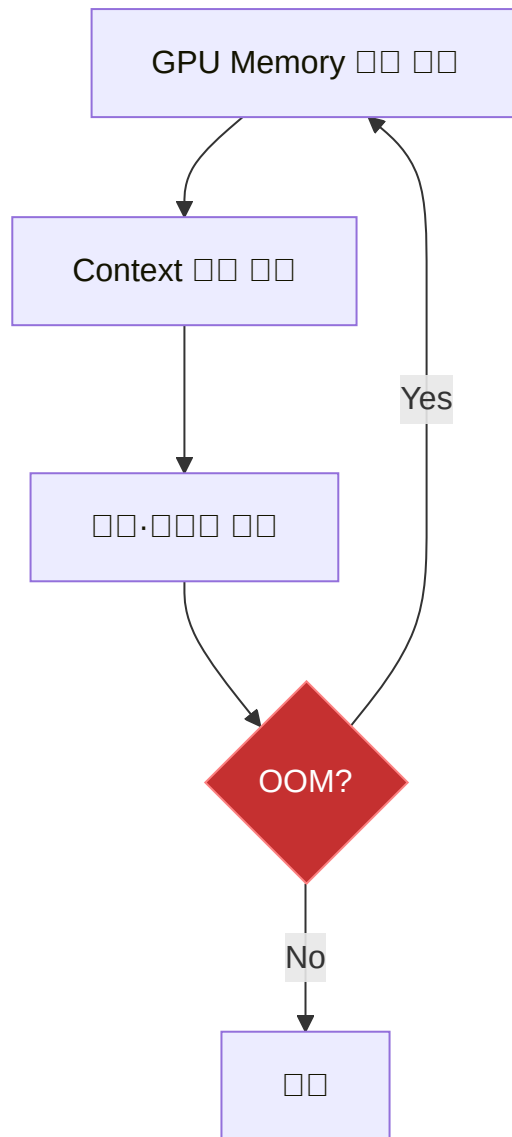
3.2 계층별 역할

계층	역할	사용자가 신경 쓰지 않아도 되는 것
Application	작업 정의 (코드 작성, 질문, 에이전트)	—
AI Runtime OS	실행 계획 수립 · 자원 배분 · 메모리 · 컨텍스트	VRAM, ctx, KV, 모델 선택
Inference Engine	텐서 연산, 샘플링, 배치 처리	엔진 내부 최적화
Hardware	연산 · 저장	물리 배치

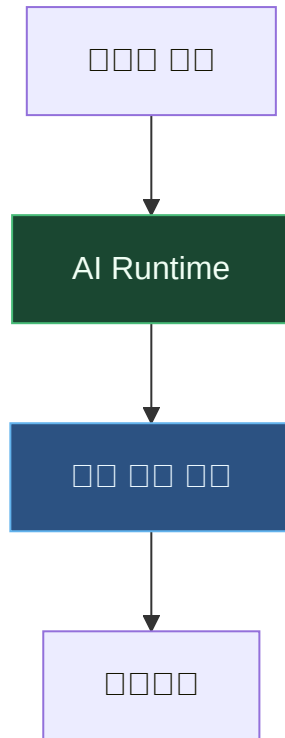
4. 핵심 철학

4.1 패러다임 전환

기존: 하드웨어를 이해해야 모델을 쓸 수 있다.



AI Runtime OS: 작업만 전달하면 실행 계획은 런타임이 만든다.



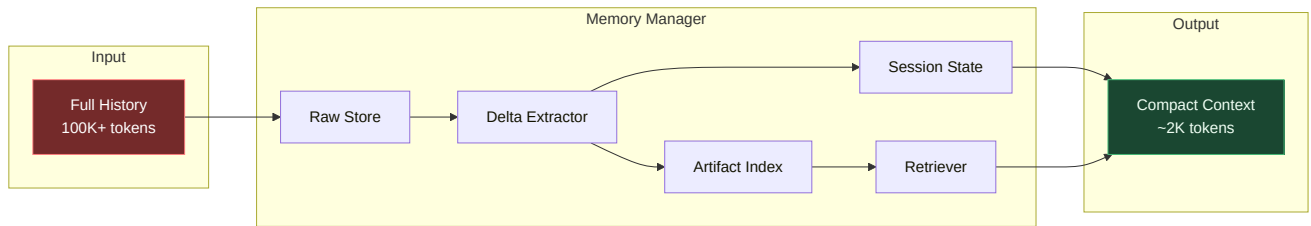
4.2 설계 원칙

원칙	설명
Task-first	사용자는 "무엇을 할지"만 말한다
OpenAI 호환	기존 IDE·에이전트를 코드 변경 없이 연결
측정 기반	벤치마크·flow trace로 개선 여부를 수치로 검증
점진적 통합	단일 모델 POC → 멀티 모델 → 멀티 GPU
Fail-safe	자동 fallback, health check, 프로필 전환

5. 주요 책임 영역

5.1 AI Memory Manager

LLM이 모든 것을 기억하게 하지 않는다. 런타임이 기억을 관리하고 LLM에는 필요한 것만 전달한다.



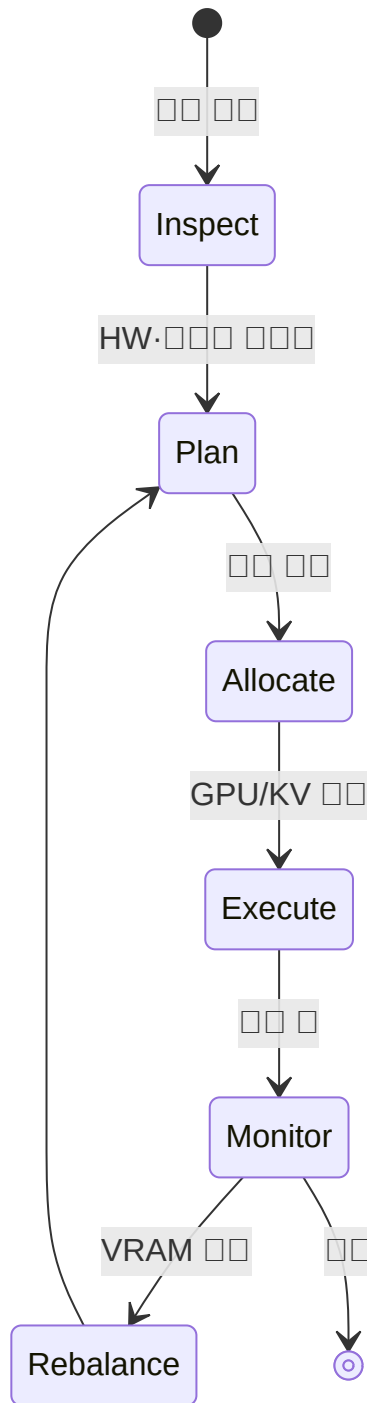
관리 대상

- 대화 메모리 (session state)
- 장기 메모리 (project-scoped)
- Delta Context (요청 간 변화량만)
- Artifact Cache (tool 결과, 파일 읽기)
- Retrieval (query/delta 점수 기반 on-demand 로드)

5.2 GPU Resource Scheduler

런타임이 지속적으로 모니터링하고 **동적으로 결정**한다.

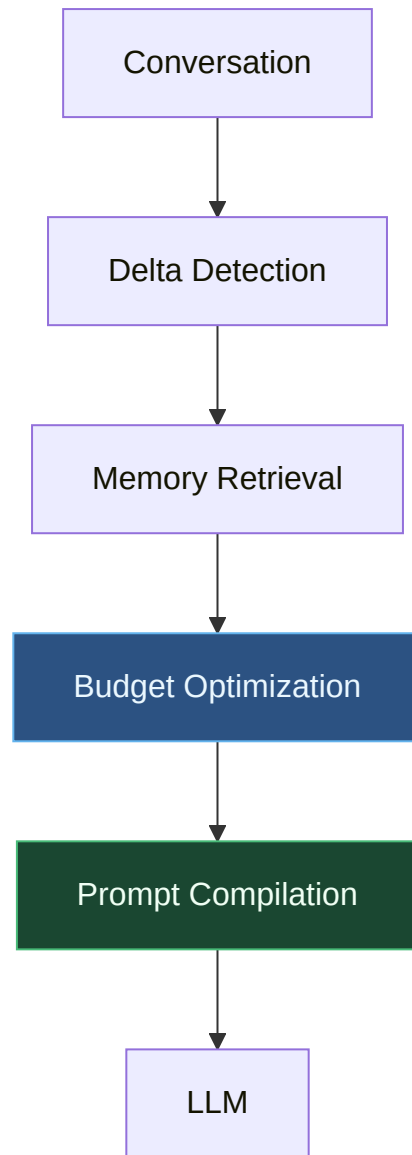
모니터링	결정
가용 VRAM / RAM	KV Cache 위치
GPU utilization	실행 GPU 선택
KV Cache 사용량	RAM offload 여부
Context 성장	컨텍스트 축소·요약
활성 Agent 수	우선순위·병렬도



비유: 현대 OS가 RAM·CPU를 스케줄링하듯, AI Runtime이 VRAM·KV-컨텍스트를 스케줄링한다.

5.3 Context Scheduler

프롬프트를 무한히 늘리지 않고 **동적으로** 구성한다.



Phase별 예산 분배 (tool_planning vs final_answer)

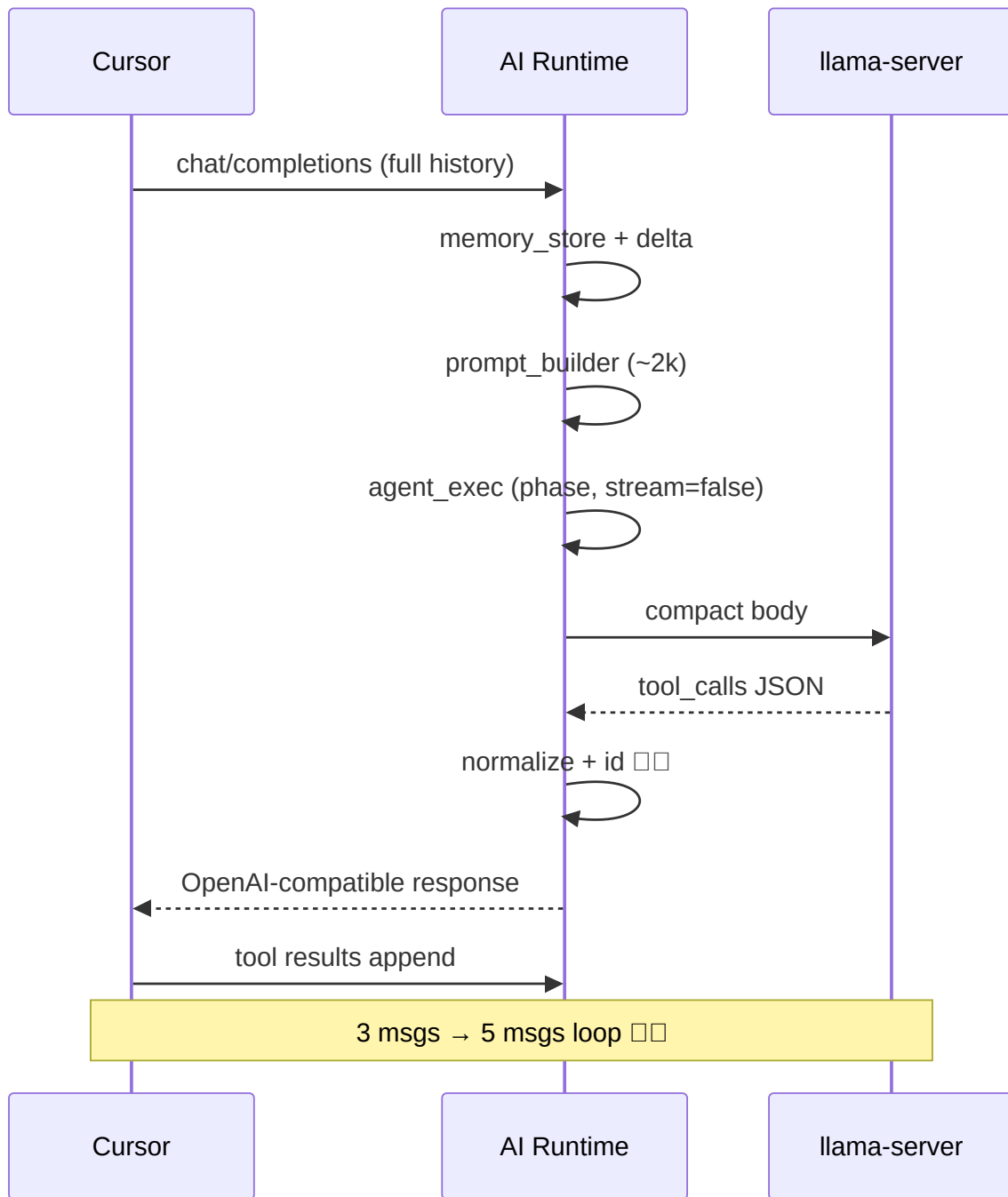
슬롯	tool_planning	final_answer
system	10%	6%
state	12%	8%
delta	8%	6%
retrieved	8%	18%
session_tail	12%	42%
task	50%	20%

Cursor가 100K+ history를내도 LLM에는 ~2K tokens만 전달 → 200K ctx 없이도 안정 운용 가능.

5.4 Agent Scheduler

여러 AI 에이전트가 제한된 하드웨어를 **경쟁**할 때 런타임이 조율한다.

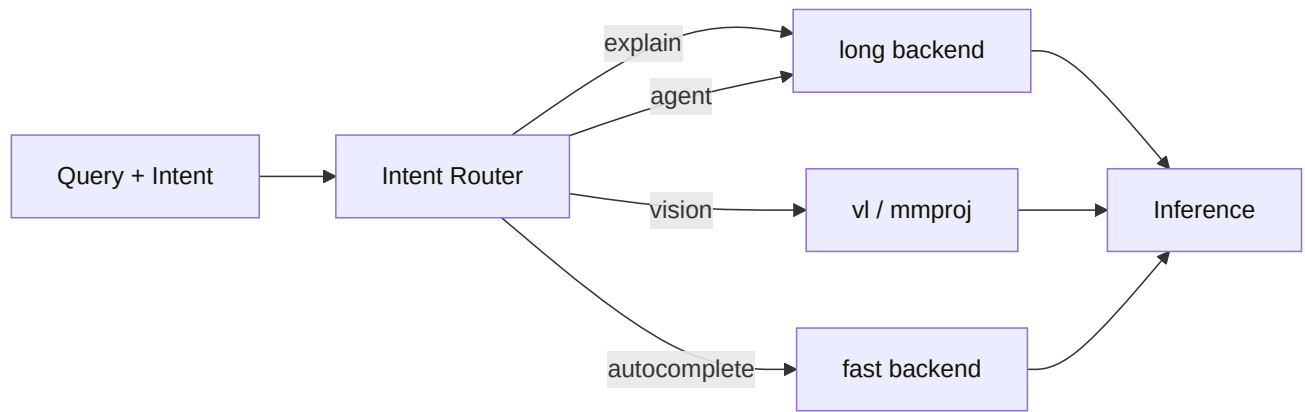
스케줄링 대상	예시
병렬 실행	background index vs foreground chat
자원 배분	VRAM quota per agent
Tool 실행	allowlist, timeout, retry
우선순위	interactive > batch
Phase 관리	tool_planning → tool_result → final_answer



5.5 Multi-Model Router

작업 특성에 맞는 모델을 자동 선택한다.

작업 유형	모델 예시
간단한 추론	7B
코드 생성	30B+ Coder
Vision	VL + mmproj
긴 문서	대용량 ctx 모델
자동완성	경량 fast 모델



효율화 원칙: 동일 weights 중복 로드 금지 (fast OFF when unified single model).

5.6 Hardware Abstraction Layer (HAL)

사용자는 다음을 알 필요가 없다.

- Tensor Split
- GPU affinity
- Context limits
- KV placement
- Quantization format

예시 하드웨어

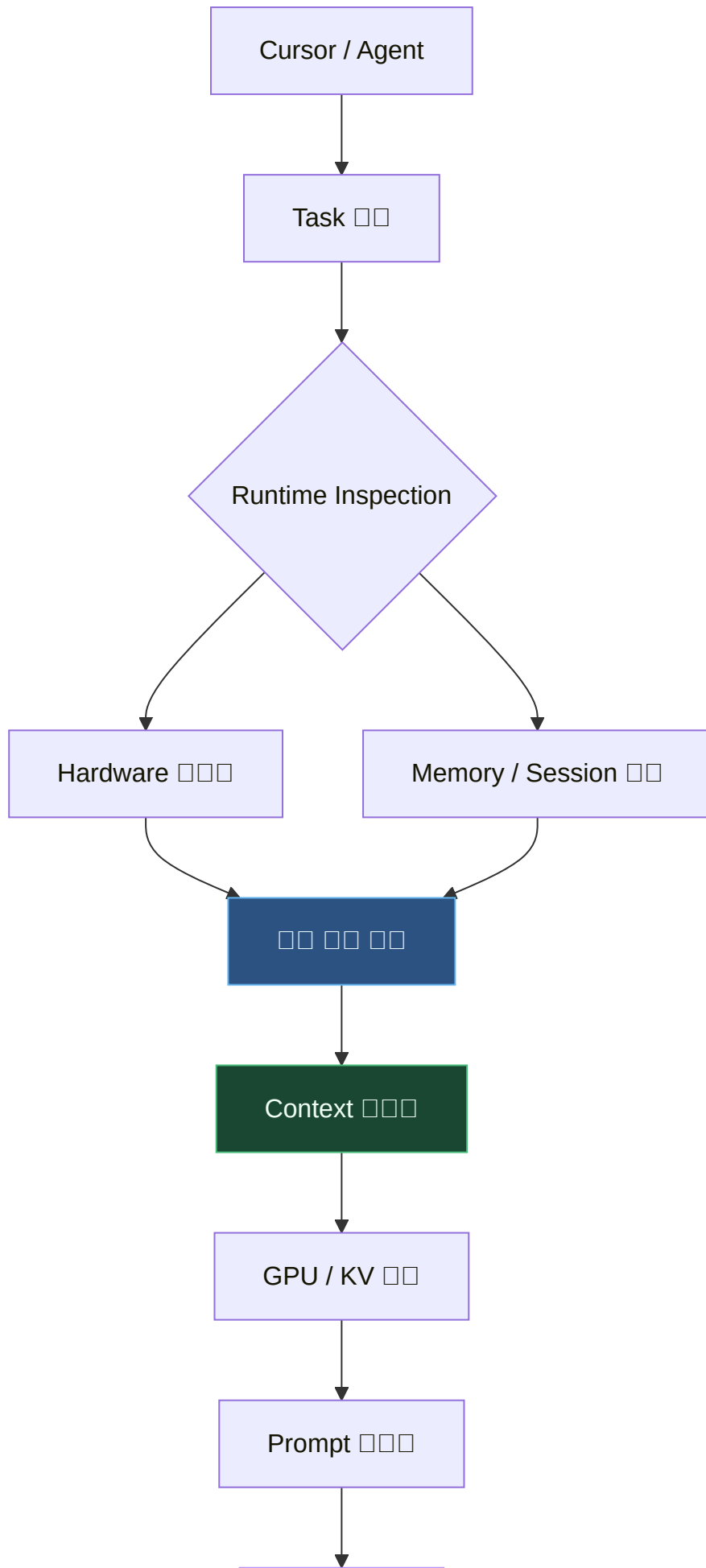
장치	스펙
GPU 1	RTX 4080 (16 GB)
GPU 2	RTX 3080 Ti (12 GB)
RAM	64 GB

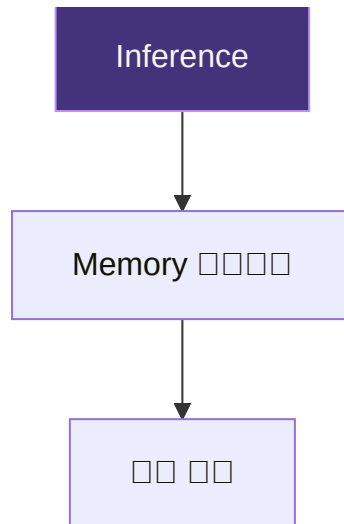
런타임이 자동 결정

- 모델 배치 (어느 GPU에 몇 layer)
 - KV allocation
 - Context budget (24K~48K 운영 / 200K 실험)
 - Memory offloading
 - Scheduling policy
-

6. 요청 실행 흐름

6.1 End-to-End





6.2 cursor-local-llm 실제 파이프라인

```
[1] Cursor IN      - full history (Router [ ] [ ])
[2] memory_store   - project / delta / artifact / session state
[3] prompt_builder - state + delta + retriever -> ~2k tokens
[4] agent_exec     - phase [ ], tool_call [ ], stream=false
[5] llama-long     - Qwen3.6 (+ mmpoj if image)
[6] Cursor        - tool append -> agent loop
```

핵심: 모든 요청은 "프롬프트 하나"가 아니라 스케줄링 문제다.

7. 구현 현황 (cursor-local-llm)

`cursor-local-llm` 은 AI Runtime OS의 첫 번째 참조 구현(POC) 이다.
Cursor + llama.cpp 환경에서 Memory·Context·Agent 실행 계층을 검증한다.

7.1 컴포넌트 매핑

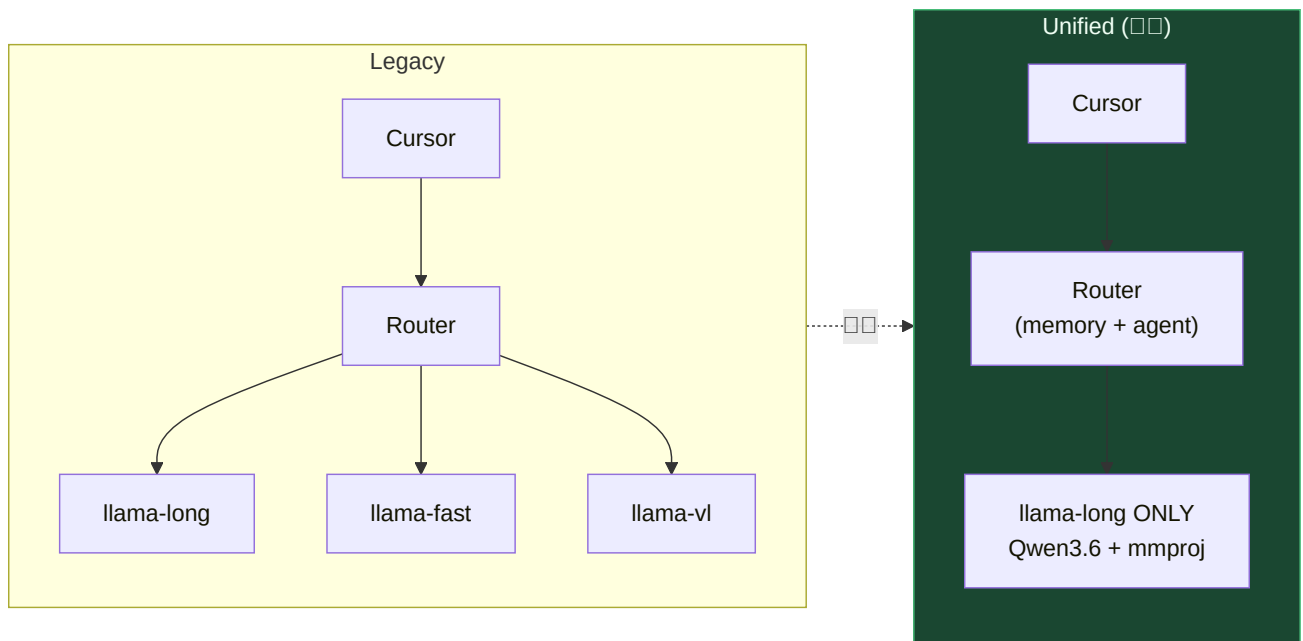
Runtime OS 컴포넌트	구현 모듈	상태
Memory Manager	memory_store.py	☐
Delta Engine	memory_store.py (delta extractor)	☐
Artifact Cache	memory_store.py + retriever.py	☐
Context Scheduler	prompt_builder.py , context_budget.py	☐
Prompt Compiler	prompt_builder.py , intent_router.py	☐
Agent Scheduler	agent_exec.py	☐
Model Router	intent_router.py , main.py	☐ 축소 (long 고정)
Tool Runtime	agent_exec.py (allowlist shell)	☐ 부분
Resource Monitor	flow_trace.py , capture.py	☐ 관측만
GPU Scheduler	docker compose, switch-model.sh	☐ 수동
KV Cache Manager	—	☐ 미구현
Multi-GPU Orchestrator	llama.cpp tensor split (수동 .env)	☐ 수동
Observability / Benchmark	benchmark-cursor-agent.py	☐

범례: ☐ 동작 · ☐ 부분 · ☐ 미구현/수동

7.2 검증된 성과

지표	Before	After (목표)
LLM 입력 tokens	full history (10K~100K+)	~2K (state+delta)
Context window 운영	200K 시도 → OOM 위험	24K~48K 안정
Agent loop	stream=true 시 3 msgs에서 중단	stream=false + tool_call id
VRAM	long + fast + vl 중복	long 단일 (unified)

7.3 아키텍처 진화



8. 로드맵

Phase 0 — POC (현재) □

- ☒ OpenAI-compatible Router proxy
- ☒ memory_store + delta + session state
- ☒ state+delta prompt_builder (~2k ctx)
- ☒ context_budget (phase별 슬롯)
- ☒ retriever (artifact on-demand)
- ☒ agent_exec (phase, stream policy, tool_call normalize)
- ☒ flow capture + benchmark

Phase 1 — 안정화 (진행 중)

- ☐ Qwen3.6 unified single-server 벤치 통과
- ☐ Vision (mmproj) 경로 검증
- ☐ 자동 fallback (tool_match < 75%, health fail)
- ☐ Cursor agent loop 3→5 msgs 안정화

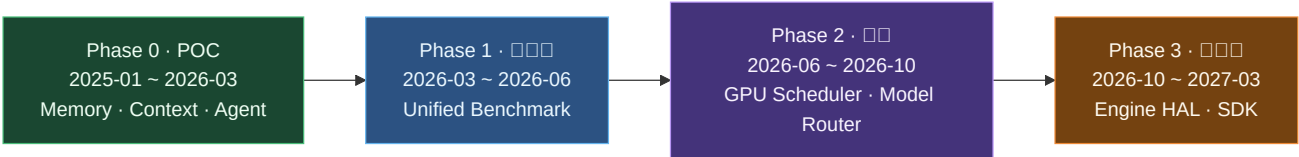
Phase 2 — Runtime 확장

- ☐ GPU Scheduler (VRAM snapshot → 동적 정책)
- ☐ KV Cache Manager (offload/reuse)
- ☐ Multi-model router (fast/long/vl 조건부, 중복 방지)

- ☐ Resource dashboard (Management Console 초안)

Phase 3 — 플랫폼화

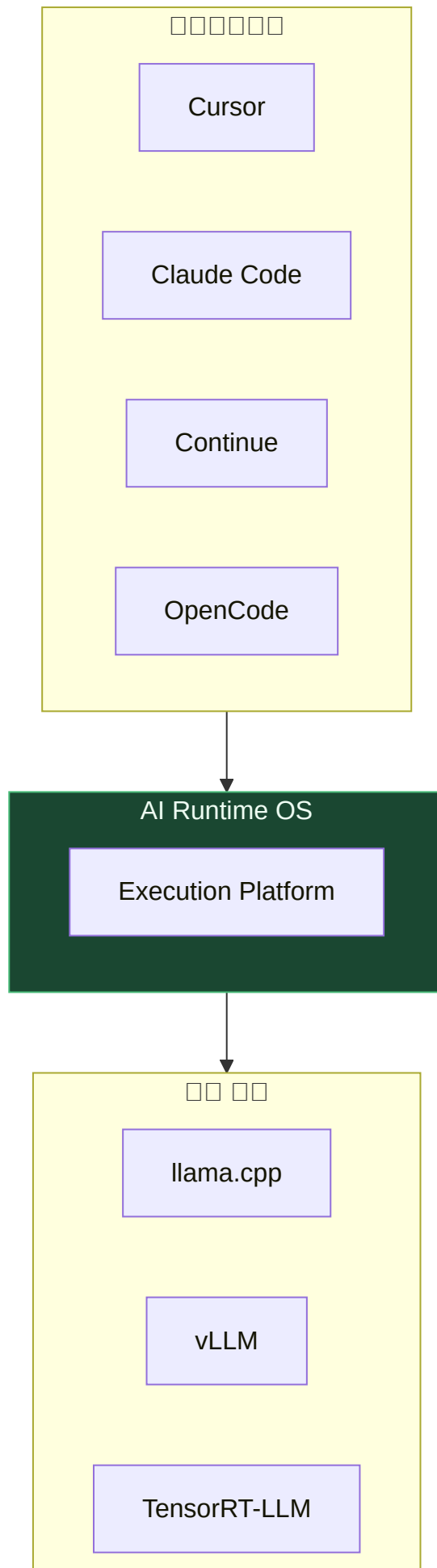
- ☐ 추론 엔진 추상화 (llama.cpp / vLLM / SGLang)
- ☐ Multi-GPU Orchestrator
- ☐ AI Runtime SDK (임베드 가능 API)
- ☐ Enterprise on-premise packaging

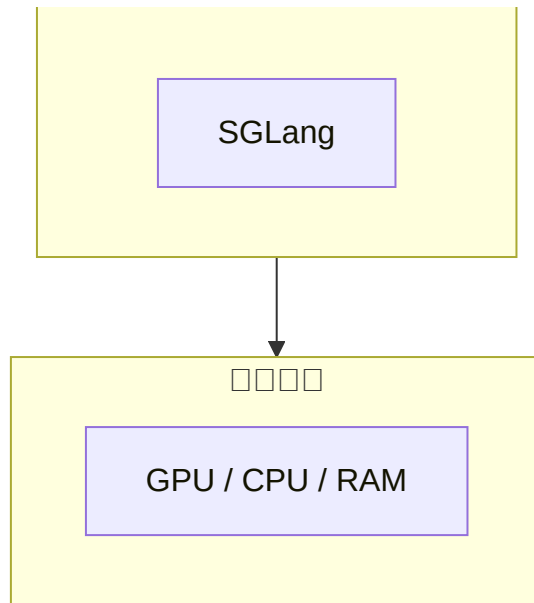


9. 경쟁 환경

제품	Agent	GPU 스케줄링	Context 스케줄링	Memory	Runtime
Cursor	☐	×	△	내부 구현	×
Claude Code	☐	×	×	△	×
OpenHands	☐	×	×	△	△
LangGraph	△	×	×	☐	△
llama.cpp	×	일부	×	×	추론 전용
vLLM	×	일부	×	×	추론 전용
AI Runtime OS	☐	☐	☐	☐	☐

9.1 포지셔닝





IDE도 아니고, 추론 엔진도 아니다. 둘 사이의 실행 플랫폼이다.

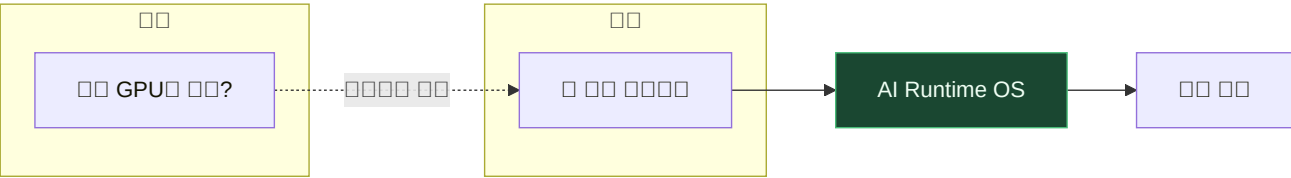
10. 비즈니스 모델

제품	설명
Enterprise On-Premise Runtime	폐쇄망·자체 GPU 환경 배포
AI Runtime SDK	앱·에이전트 프레임워크에 임베드
AI Runtime Enterprise	SLA, 멀티 테넌트, 정책 엔진
Runtime Management Console	VRAM·에이전트·모델 모니터링 UI
Commercial Support	엔터프라이즈 기술 지원
Embedded Runtime Licensing	OEM / appliance 라이선스

11. 장기 비전

오늘의 OS는 애플리케이션을 관리한다.

내일의 AI 시스템은 지능(intelligence) 자체를 관리하는 소프트웨어가 필요하다.



AI Runtime OS는 그 기반 계층이 되는 것을 목표로 한다.

12. 부록: 효율화 전략

비전 문서에 없었지만, 현재 POC에서 검증·권장되는 효율화 방안이다.

12.1 Context: "크게 열지 말고, 작게 보내라"

전략	효과
State + Delta 프록시	80~90% context 절감
Phase별 budget (context_budget.py)	tool vs answer 슬롯 분리
Retriever on-demand	필요한 artifact만 로드
24K~48K 운영 ctx	KV VRAM 안정, OOM 방지

핵심 인사이트: ctx를 키우는 것보다 보내는 양을 줄이는 것이 더 효과적이다.

12.2 VRAM: "모델 하나, 역할 여러 개"

전략	효과
Unified single server (Qwen3.6 + mmproj)	fast/vl 중복 제거
프로필 기반 fallback	qwen3_6 ↔ qwen3_coder
long만 기동 (fast/vl OFF)	VRAM 2~3배 절약

12.3 Agent: "프로토콜 호환이 성능이다"

전략	효과
agent/debug → <code>stream=false</code>	Cursor tool loop 안정
<code>ensure_tool_call_ids()</code>	tool_call id 누락 방지
tool_call JSON/XML 정규화	leak 0, json_valid $\geq$ 90%
phase 감지 (tool_planning / final_answer)	슬림 system prompt

12.4 관측: "측정 없이 최적화 없다"

도구	용도
<code>benchmark-cursor-agent.py</code>	tool_match, json_valid, leaks, tps
<code>flow_trace.py</code> / <code>cursor-captures</code>	요청별 delta·token 추적
<code>handoff.md</code>	단계별 완료·다음 작업 기록

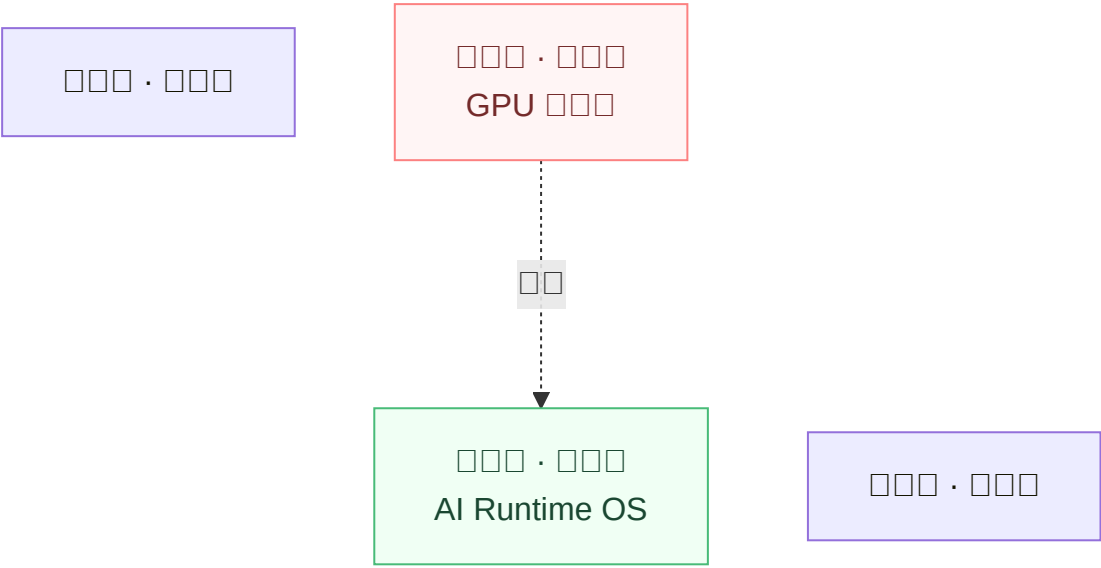
권장 통과 게이트

- `tool_match_rate` $\geq$ baseline
- `json_valid_rate` $\geq$ 90%
- `xml_leaks` = 0
- `ctx_used_tokens` $\approx$ 2K (full history 대비)

12.5 아직 부족한 영역 (추가 권장)

영역	권장 방향
KV Cache 재사용	동일 session prefix cache hit (llama.cpp prompt cache API)
동적 모델 언로드	idle timeout 후 VRAM 반환
우선순위 큐	interactive vs background agent
Semantic retrieval	키워드 → embedding 기반 artifact 검색
Cross-project memory	project_key 스코프 + 선택적 공유
Health-driven routing	latency/OOM 자동 프로필 전환
Management UI	VRAM·ctx·agent phase 실시간 대시보드

12.6 왜 이것이 중요한가



대부분의 팀은 더 큰 GPU로 스케일한다.
AI Runtime OS는 기존 하드웨어를 더 똑똑하게 쓰는 방식으로 스케일한다.

기대 효과	수치 (목표)
Context 절감	80~90%
Agent 성공률	tool loop 안정화
GPU 활용률	중복 모델 제거
VRAM 소비	단일 unified 모델
소비자급 GPU	더 큰 모델 운용 가능

관련 문서

문서	내용
ARCHITECTURE.md	cursor-local-llm 통합 아키텍처 (Qwen3.6)
WORKSPACE_LAYOUT.md	디렉토리·워크스페이스 규칙
../handoff.md	최근 구현 상태·다음 작업

Last updated: 2026-06-18